

✓ Welcome to Colab!

1. Why NumPy is Used

Python lists are flexible but slow for numerical calculations. NumPy provides:

Fast array operations Memory-efficient storage Mathematical functions Linear algebra operations
Random number generation Statistical calculations Support for scientific libraries

Common users:

Data scientists Machine learning engineers Researchers Engineers Financial analysts

2. Installing NumPy

Using pip:

```
pip install numpy
```

Using Anaconda:

```
conda install numpy
```

Importing:

```
import numpy as np
```



3. NumPy Array

The main object in NumPy is the ndarray (N-dimensional array).

Example:

```
import numpy as np

a = np.array([1, 2, 3, 4])
```

```
print(a)
```

Output:

```
[1 2 3 4]
```

Unlike Python lists, NumPy arrays:

Store elements of the same data type Support vectorized operations Are faster for calculations

4. Creating Arrays From Python List

Start coding or [generate](#) with AI.

```
np.array([1,2,3])
```

2D Array

```
np.array([[1,2],[3,4]])
```

Output:

```
[[1 2]
 [3 4]]
```

Zeros

```
np.zeros((3,3))
```

Creates:

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

Ones

```
np.ones((2,4))
```

Identity Matrix

```
np.eye(3)
```

Output:

```
[[1. 0. 0.]  
 [0. 1. 0.]  
 [0. 0. 1.]]
```

5. Range Functions arange()

Similar to Python range:

linspace()

```
np.linspace(0,1,5)
```

Output:

```
[0. 0.25 0.5 0.75 1.]
```

6. Array Properties

Example:

Double-click (or enter) to edit

```
a = np.array([[1,2,3],[4,5,6]])
```

Shape

```
a.shape
```

Output:

```
2
```

Size

```
a.size
```

Output:

```
6
```

Data Type

```
a.dtype
```

Output:

int64

7. Data Types

Common NumPy types:

Type Meaning int32 32-bit integer int64 64-bit integer float32 32-bit decimal float64 64-bit decimal
bool True/False complex Complex numbers

Example:

```
np.array([1,2,3], dtype=float)
```

8. Indexing

1D:

```
a=np.array([10,20,30])  
a[0]
```

Output:

10

2D:

```
a=np.array([[1,2],[3,4]])  
a[0,1]
```

Output:

2

9. Slicing

```
a=np.array([1,2,3,4,5])  
a[1:4]
```

Output:

[2 3 4]

Reverse:

```
a[::-1]
```

Output:

[5 4 3 2 1]

10. Array Operations Addition

```
a=np.array([1,2,3])  
b=np.array([4,5,6])  
  
a+b
```

Output:

[5 7 9]

Multiplication

```
a*b
```

Output:

[4 10 18]

Division

```
b/a
```

11. Mathematical Functions Square Root

```
np.sqrt([4,9,16])
```

Output:

[2. 3. 4.]

Power

```
np.power(2,3)
```

Output:

8

Exponential

```
np.power(2,3)
```

Logarithm

```
np.log([1,10,100])
```

12. Statistical Functions Mean

```
np.mean([1,2,3,4])
```

Median

```
np.median([1,2,3,4])
```

Standard Deviation

```
np.std([1,2,3,4])
```

Variance

```
np.var([1,2,3,4])
```

Minimum

```
np.min([5,2,9])
```

Maximum

```
np.max([5,2,9])
```

13. Matrix Operations

Matrix multiplication:

```
a=np.array([[1,2],[3,4]])  
b=np.array([[5,6],[7,8]])  
  
np.dot(a,b)
```

14. Transpose

```
a.T
```

Example:

Before:

```
[[1 2] [3 4]]
```

After:

```
[[1 3] [2 4]]
```

15. Reshaping Arrays

Example:

```
a=np.arange(12)  
  
a.reshape(3,4)
```

Output:

```
[[0 1 2 3]  
 [4 5 6 7]  
 [8 9 10 11]]
```

16. Flattening Arrays

```
a.flatten()
```

Example:

```
[[1,2],[3,4]]
```

17. Joining Arrays Concatenate

```
np.concatenate((a,b))
```

Stack

```
np.vstack((a,b))
```

```
np.hstack((a,b))
```

18. Splitting Arrays

```
np.split(array,2)
```

19. Random Module

```
np.random.rand(3)
```

Output:

```
[0.2 0.8 0.5]
```

Random integers:

```
Random integers:
```

Normal distribution:

```
np.random.randn(5)
```

Set seed:

```
np.random.seed(42)
```

20. Sorting

```
a=np.array([3,1,2])
```

```
np.sort(a)
```

Output:

[1 2 3]

21. Searching

```
np.where(a==2)
```

22. Copy vs View

```
b=a.copy()
```

View

```
b=a.view()
```

23. Saving and Loading Data

```
np.save("data.npy", a)
```

Load:

```
np.load("data.npy")
```

CSV:

```
np.savetxt("data.csv", a)
```

24. Broadcasting

Broadcasting allows operations between different shapes.

Example:

```
a=np.array([1,2,3])  
  
a+5
```

Output:

[6 7 8]

25. Linear Algebra Module

```
import numpy.linalg as la
```

Functions:

Determinant

```
la.det(a)
```

Inverse

```
la.inv(a)
```

Eigenvalues

```
la.eig(a)
```

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

(2,3)

Number of Dimensions

```
Start coding or generate with AI.
```

conclusion

NumPy is a powerful and efficient Python library for numerical computing. It provides fast multidimensional arrays, mathematical operations, statistical functions, linear algebra, random number generation, and array manipulation. Compared to Python lists, NumPy is faster, uses less memory, and makes data processing easier. It is widely used in data science, machine learning, artificial intelligence, scientific computing, and data analysis, making it one of the most important libraries in the Python ecosystem.

